

Personal Research Assistant

A Multi-Agent System Design
Project

Presented by:
Noura Abuthnain



System Overview

The **Personal Research Assistant** is a multi-agent AI system designed to assist a **single knowledge worker** through a unified conversational interface. The system coordinates specialized agents to retrieve information from the user's personal knowledge base, conduct external research, manage workspace files, and generate structured reports. A central orchestrator agent plans the workflow, delegates tasks to the appropriate agents, and combines their outputs into a single, coherent response.

The system supports five primary capabilities:

- **General Assistance:** Answer general questions and maintain natural conversations.
- **Knowledge Retrieval:** Search the user's personal notes and documents with grounded citations.
- **External Research:** Retrieve and summarize information from trusted external sources.
- **Workspace Management:** Safely create, read, update, and organize files within a sandboxed workspace.
- **Report Generation:** Produce structured research reports and save them to user-specified locations.

Agent Architecture

The Personal Research Assistant is implemented using a small set of specialized agents. Following the design principle of **using the fewest agents necessary**, the General Assistant and Orchestrator are combined into a single Executive Agent, while **Report Generation** is implemented as a capability rather than a standalone agent. This reduces coordination overhead while maintaining clear responsibilities.

Agent	Responsibilities
Executive Agent	Plans tasks, routes requests, validates execution, and generates the final response.
Knowledge Agent	Retrieves grounded information from the user's personal knowledge base with citations.
Research Agent	Searches trusted external sources and extracts cited information.
Workspace Agent	Performs secure file operations within the sandboxed workspace.

Architecture Topology

The system adopts a **supervisor topology**, where a central orchestrator agent **receives user requests**, creates an execution plan, delegates tasks to specialist agents, and combines their outputs into a single response. **This topology was selected because** it provides centralized control, modularity, and flexible coordination while maintaining a unified user experience.

Trade-off: The supervisor introduces a **single coordination point and a small routing overhead**. However, this trade-off is acceptable because it simplifies task management, error handling, and future scalability.

Topology	Reason
Single	Limited specialization
Sequential	Too rigid for dynamic tasks
Network	Complex coordination
Supervisor	Best balance of control, flexibility, and scalability

Routing Strategy

The orchestrator uses a **hybrid routing strategy**, combining LLM-based intent detection with rule-based validation. The LLM identifies **the user's intent and selects the required agent(s)**, while predefined rules validate permissions, **file paths, and execution order before tasks are executed**.

Trade-off: Hybrid routing is more flexible and accurate than fixed rules alone, but it **adds a small validation step before execution**. This trade-off improves reliability and prevents incorrect or unsafe routing.

Execution Flow

The system uses a mixed execution model. Dependent tasks, such as **research → report generation → file writing**, run sequentially. Independent tasks, such as searching the personal knowledge base and external sources, **may run in parallel** to reduce response time. The orchestrator synchronizes and merges all results before generating the final response.

Trade-off: Parallel execution improves performance **but increases the complexity of coordination and state management**. Therefore, only independent retrieval tasks run in parallel, while dependent and write operations remain sequential to ensure consistency.

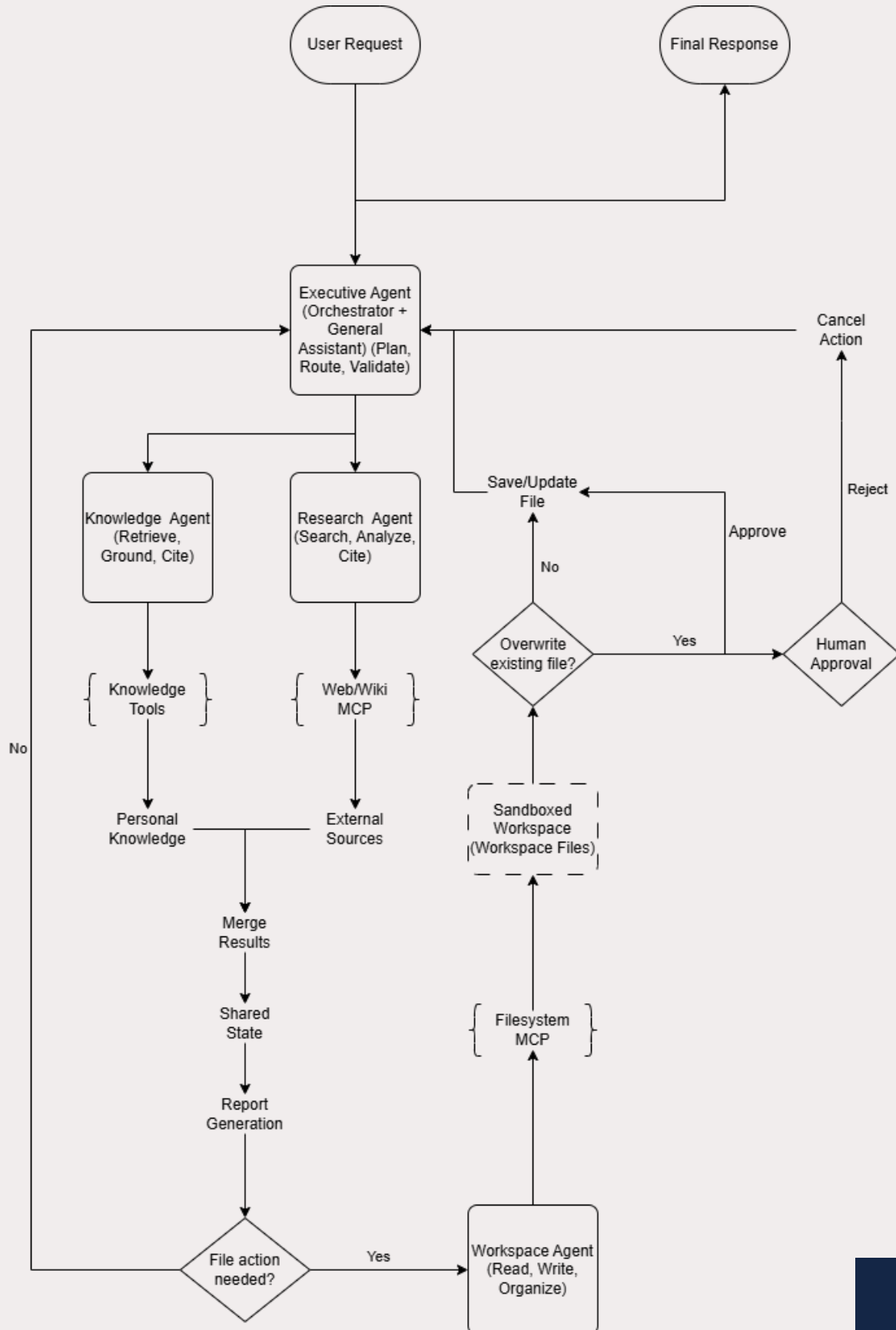
Execution Lifecycle

The orchestrator stops execution when the requested task is **completed**, the maximum number of execution steps is **reached**, the error budget is **exceeded**, user confirmation is **required**, or an unrecoverable error **occurs**.

Trade-off: Strict stop conditions improve reliability and prevent unnecessary execution, **but they may terminate the workflow** before all optional tasks are completed. This trade-off ensures predictable behavior and protects system resources.

Stop Condition	Action
Goal achieved	Return final response
Maximum steps reached	Stop execution
Error budget exceeded	Terminate workflow
User confirmation required	Pause execution
Critical failure	Return error

System Workflow



Shared State Management

The system maintains a shared state that **stores information exchanged between agents**. Each agent reads only the fields it needs and updates only its assigned outputs. Independent fields use append operations, while single-owner fields use replace operations to prevent conflicts.

Field	Updated By	Merge Strategy
messages	All agents	Append
plan	Orchestrator	Replace
research_findings	Research Agent	Append
citations	Knowledge and Research	Append
draft_report	Report Generation	Replace
file_path	User / Orchestrator	Replace

State Persistence and Human Oversight

The shared state is persisted using a **checkpoint store**, allowing the workflow to resume after failures and maintain memory across multiple turns. Each conversation is **assigned a unique thread ID**, ensuring that messages, plans, and intermediate results remain isolated from other user conversations.

Trade-off: Persisting state improves reliability and enables recovery after interruptions, **but it requires additional storage and state management.**

Human approval is **required only for high-risk actions, such as overwriting an existing file or modifying important workspace data.** Read-only operations, knowledge retrieval, and external research execute automatically without interruption.

Trade-off: Human approval slightly increases task completion time but prevents accidental data loss and improves system safety.

Communication Architecture

Agents communicate using **typed, structured messages** instead of free text. Typed messages ensure that every component receives predictable fields, allow automatic validation, reduce ambiguity, and simplify error handling, logging, and implementation.

Trade-off: Structured messages require more initial design than free text, but they improve reliability and make agent interactions easier to test and debug.

```
class AgentMessage(BaseModel):  
    id: str  
    sender: str  
    recipient: str  
    type: Literal["request", "response", "handoff", "error"]  
    payload: dict[str, Any]  
    timestamp: datetime
```

Message Structure

Interaction	Pattern	Reason
Executive Agent → Knowledge Agent	Request / Response	Retrieve grounded knowledge.
Executive Agent → Research Agent	Request / Response	Retrieve external information.
Research Agent → Report Generation	Handoff	Transfer research results.
Executive Agent → Workspace Agent	Request / Response	Perform file operations safely.
All Components → Shared State	Blackboard	Share execution state.
Broadcast	Not Used	Unnecessary with a supervisor topology.

Message Structure

Example Request Message:

```
{
  "id": "msg-101",
  "sender": "ExecutiveAgent",
  "recipient": "ResearchAgent",
  "type": "request",
  "payload": {
    "task": "Research the top three vector databases",
    "criteria": [
      "features",
      "deployment options",
      "strengths",
      "limitations"
    ],
    "required_sources": 3,
    "output_format": "structured_findings"
  },
  "timestamp": "2026-07-11T12:00:00+03:00"
}
```

Message Structure

Matching Response Message:

```
{
  "id": "msg-102",
  "sender": "ResearchAgent",
  "recipient": "ExecutiveAgent",
  "type": "response",
  "payload": {
    "request_id": "msg-101",
    "status": "completed",
    "research_findings": [
      {
        "name": "Database A",
        "summary": "Structured research finding."
      },
      {
        "name": "Database B",
        "summary": "Structured research finding."
      },
      {
        "name": "Database C",
        "summary": "Structured research finding."
      }
    ],
    "citations": [
      {
        "title": "Source 1",
        "url": "source-reference-1"
      },
      {
        "title": "Source 2",
        "url": "source-reference-2"
      }
    ]
  },
  "timestamp": "2026-07-11T12:00:08+03:00"
}
```

SECTION 3.4

Agent Tools

Each agent receives **only the tools required for its role**, reducing unnecessary access and security risk.

Component	Tool	Description	Risk
Executive Agent	validate_plan	Validate execution plan	read
Knowledge Agent	search_knowledge	Search personal knowledge	read
	read_knowledge_item	Retrieve document content	read
Research Agent	search_web	Search external sources	network
	fetch_article	Retrieve article content	network
Workspace Agent	list_files	List workspace files	read
	read_file	Read workspace file	read
	create_file	Create new file	write
	update_file	Update existing file	write
Report Generation	No external tool	Generate report	—

MCP Architecture

Two capabilities are exposed as MCP servers:

- Web/Wiki MCP Server for external research
- Filesystem MCP Server for workspace operations

MCP is selected because these capabilities are external, reusable, and require clear permission boundaries. The personal knowledge tools remain local because they access a small private index used only by the Knowledge Explorer.

Trade-off: MCP adds configuration and communication overhead, but improves modularity, security, and tool reuse.

MCP Server	Tools	Resources	Client
Web/Wiki MCP	search_web, fetch_article	Search results, article content, titles, URLs, and metadata	Research Agent
Filesystem MCP	list_files, read_file, create_file, update_file	Files and folders inside the approved workspace sandbox	Workspace Agent

Prompting Strategy

Each component uses prompting techniques that match its responsibilities. The executive agent focuses on planning and routing, while specialist agents focus on retrieval, grounding, and safe tool usage.

Component	Prompting Technique	Purpose
Executive Agent	Role, Task Decomposition, Structured Output	Plan, route, and coordinate tasks.
Knowledge Agent	Grounding, Source Citation	Retrieve accurate, cited information.
Research Agent	ReAct, Source Citation	Search, reason, and cite sources.
Workspace Agent	Guardrails, Structured Output	Perform safe file operations.
Report Generation	Persona, Few-shot	Produce consistent reports.

Agents Prompt

Executive Agent Prompt

You are the Executive Agent of a Personal Research Assistant.

Your responsibilities are:

- Understand the user's request.
- Break complex tasks into smaller steps.
- Select the required specialist agents.
- Produce a structured execution plan.
- Never answer using unsupported information.
- Return the final response after collecting all results.

Research Agent Prompt

You are the Research Agent.

Your responsibilities are:

- Search trusted external sources.
- Extract only relevant information.
- Cite every source used.
- Do not invent facts.
- Return structured findings to the Executive Agent.

References

[1] Anthropic, "Model Context Protocol Documentation." [Online]. Available: <https://modelcontextprotocol.io/introduction>

[2] Pydantic, "Pydantic Documentation." [Online]. Available: <https://docs.pydantic.dev/latest/>

[3] OpenAI, "Prompt Engineering Guide." [Online]. Available: <https://platform.openai.com/docs/guides/prompt-engineering>

[4] OpenAI, "Function Calling Guide." [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>



PERSONAL RESEARCH ASSISTANT